
Pulizia totale di un account Bluesky con skeeter-deleter

Guida pratica, passo per passo, alla cancellazione bulk di post, repost e like su Bluesky, con gli errori realmente incontrati durante un'operazione su un account di circa 2.600 elementi e le soluzioni applicate.

A cura di Claudio · Luglio 2026

Guida pratica per cancellare **tutti i post, repost e like** da un account Bluesky usando [skeeter-deleter](#), un tool CLI open source basato sulle API ufficiali AT Protocol.

Questa guida nasce da un'operazione reale su un account con circa 2.600 elementi tra post, repost e like. Include gli errori effettivamente incontrati durante il processo e le soluzioni applicate, non solo la teoria.

Prima di iniziare, leggi questo: l'operazione è **distruttiva e irreversibile**. Una volta cancellato, un post non è recuperabile. Se vuoi un backup, sappi che lo script ne crea uno automaticamente durante l'esecuzione (vedi Capitolo 4) — ma se non ti interessa conservare nulla, puoi ignorarlo.

Nota pratica: questo PDF è pensato per la lettura. Per copiare rapidamente i comandi mentre lavori nel Terminale, consulta la versione Markdown di questa guida direttamente su GitHub — ogni blocco di codice ha un pulsante “copia” integrato.

Compatibilità: questa guida è stata scritta e testata su macOS. Gli script Python e i comandi Git/pip sono cross-platform e dovrebbero funzionare senza modifiche su Linux. Su Windows, la strada più semplice è usare WSL (Windows Subsystem for Linux), che replica un ambiente Linux — la guida non è stata testata in quel contesto, ma i passaggi Python dovrebbero valere allo stesso modo. I comandi specifici per la gestione dello stand-by (Capitolo 5) sono invece solo per macOS.

Indice

1. Operazioni preliminari _____
2. Setup dell'ambiente _____
3. Configurazione delle credenziali _____
4. Come funziona lo script: le fasi _____
5. Gestione dello stand-by del Mac _____
6. Primo lancio e interpretazione dell'output _____
7. Problema noto: rate limit non gestito correttamente _____
8. Soluzione: script modificato senza download media _____
9. Verifica finale (non fidarsi della barra di progresso) _____

10. Pulizia dei residui isolati

11. Ripristino del sistema

12. Considerazioni per account con molti più elementi

1. Operazioni preliminari

1.1 Crea un App Password su Bluesky

Non usare mai la password principale del tuo account per uno script di terze parti. Bluesky permette di generare password dedicate, revocabili singolarmente:

1. Vai su **Impostazioni** → **Privacy e sicurezza** → **App Passwords**
2. Crea una nuova password, dandole un nome riconoscibile (es. `skeeter-deleter`)
3. Copia la password generata (formato tipo `xxxx-xxxx-xxxx-xxxx`) e conservala temporaneamente: ti servirà al Capitolo 3

1.2 Verifica di non dover restare loggato da nessuna parte

Lo script comunica direttamente con le API Bluesky usando le credenziali che gli fornirai. **Non serve** restare loggati nel browser o nell'app durante l'esecuzione: puoi chiudere tutto, lo script è indipendente da qualsiasi sessione.

2. Setup dell'ambiente

Serve un Mac (o sistema Unix-like) con Python 3 e Git.

2.1 Verifica Python e Git

```
python3 --version
git --version
```

Se mancano, Python si scarica da python.org o via Homebrew; Git viene proposto automaticamente da macOS al primo utilizzo (Xcode Command Line Tools).

Nota sulla versione di Python: se usi Python 3.14 o più recente, alcune dipendenze del progetto potrebbero avere problemi di compatibilità — vedi passaggio 2.4.

2.2 Clona il repository

```
cd ~
git clone https://github.com/Gorcenski/skeeter-deleter.git
cd skeeter-deleter
```

2.3 Installa le dipendenze base

```
pip3 install -r requirements.txt --break-system-packages
```

2.4 Problema noto: `atproto` incompatibile con Python 3.14+

Il file `requirements.txt` del progetto fissa `atproto==0.0.54`, una versione che **non supporta Python 3.14+** (richiede `<3.12` o `<3.13` a seconda della sotto-versione). Se ricevi un errore tipo:

```
ERROR: Could not find a version that satisfies the requirement atproto==0.0.54
```

Risolvi installando la versione più recente disponibile di `atproto` (compatibile con Python 3.14), poi le restanti dipendenze:

```
pip3 install atproto --break-system-packages
pip3 install python-dateutil python-magic python-twitter-v2 rich dataclasses-json
marshmallow requests --break-system-packages
```

2.5 Problema noto: `libmagic` mancante

Il pacchetto Python `python-magic` è solo un wrapper attorno alla libreria di sistema `libmagic`, che va installata separatamente. Se al primo avvio ricevi:

```
ImportError: failed to find libmagic. Check your installation
```

Risolvi con Homebrew:

```
brew install libmagic
```

2.6 Verifica finale

```
python3 skeeter_deleter.py --help
```

Se vedi l'elenco delle opzioni senza errori, l'ambiente è pronto.

3. Configurazione delle credenziali

Lo script legge le credenziali da variabili d'ambiente, non le chiede a runtime:

```
export BLUESKY_USERNAME="tuo-handle.bsky.social"
export BLUESKY_PASSWORD="la-tua-app-password"
```

Attenzioni: - `BLUESKY_USERNAME` è il tuo **handle** (es. `nomeutente.dominio.tld`), **senza** la `@` iniziale — non l'email di registrazione. - `BLUESKY_PASSWORD` è l'App Password generata al Capitolo 1, non la password principale dell'account. - Queste variabili valgono solo per la sessione di Terminale corrente: se chiudi il Terminale, andranno reimpostate.

4. Come funziona lo script: le fasi

Lo script esegue automaticamente, in sequenza, queste operazioni (senza possibilità di eseguirle separatamente, salvo modifiche manuali — vedi Capitolo 8):

1. **Archiviazione del repo:** scarica una copia locale (formato `.car`) di tutto il tuo repository AT Protocol, salvata in `archive/{tuo-did}/`.
2. **Download dei media:** scarica ogni immagine allegata ai tuoi post, salvandola localmente in `archive/{tuo-did}/_blob/`.
3. **Raccolta dei dati:** recupera le liste di post, like e repost da elaborare, in base ai filtri specificati.
4. **Unlike:** rimuove tutti i like che soddisfano i criteri.
5. **Delete:** rimuove post e repost che soddisfano i criteri.

Le fasi 1 e 2 servono a costruire un indice locale necessario allo script per sapere cosa cancellare — **consumano una quota significativa di chiamate API**, specialmente la fase 2 se hai molte immagini (vedi Capitolo 7).

Parametri principali

Flag	Significato	Attenzione
<code>-s N</code> , <code>--stale-limit N</code>	Cancella post più vecchi di N giorni	Default <code>0</code> = nessun limite , cioè non cancella nulla per età. Per una pulizia totale, usa un valore basso come <code>-s 1</code>
<code>-l N</code> , <code>--max-reposts N</code>	Cancella post con più di N repost (virali)	Default <code>0</code> = nessun limite
<code>-d domini</code> , <code>--domains-to-protect</code>	Whitelist di domini da non toccare	Default vuoto
<code>-y</code> , <code>--yes</code>	Salta le conferme interattive	Necessario per automazione
<code>-v</code> / <code>-vv</code>	Livello di verbosità dell'output a schermo	<code>-vv</code> mostra il dettaglio di ogni elemento processato

Nota importante sui repost: nel codice, i repost vengono raccolti da `gather_reposts()` e **aggiunti alla stessa lista dei post da cancellare** (`self.to_delete.extend(to_unrepost)`). Non esiste quindi una fase “unrepost” visibile separatamente nell'output: appare tutto insieme sotto “Deleting”.

5. Gestione dello stand-by del Mac

Il processo può durare da alcune decine di minuti a qualche ora, a seconda del volume. Se il Mac va in stand-by, la connessione si interrompe e lo script si blocca.

5.1 Prendi nota delle impostazioni attuali (per ripristinarle dopo)

```
pmset -g
```

Annota i valori di `standby`, `disksleep`, `sleep`, `displaysleep`.

5.2 Disabilita lo stand-by mentre il Mac è collegato alla corrente

```
sudo pmset -c sleep 0 disksleep 0 standby 0
```

Il flag `-c` applica la modifica solo quando il Mac è sotto carica — se lo scolleghi, torna al comportamento normale a batteria.

Importante: collega il Mac all'alimentazione per tutta la durata dell'operazione. Su batteria, l'impostazione `-c` non ha effetto e il Mac potrebbe comunque entrare in sospensione.

5.3 Alternativa più leggera: `caffeinate`

Per operazioni più brevi, puoi evitare di toccare le impostazioni di sistema e usare `caffeinate`, che previene lo sleep solo per la durata del comando che lo segue:

```
caffeinate -i python3 skeeter_deleter.py [parametri]
```

Per operazioni lunghe (giorni, come nel caso di account con decine di migliaia di elementi), il metodo `pmset` a livello di sistema è più robusto: `caffeinate` protegge solo la sessione di Terminale attiva, e si perde se il Terminale si chiude per qualsiasi motivo.

6. Primo lancio e interpretazione dell'output

Comando completo per una cancellazione totale (post, repost, like):

```
caffeinate -i python3 skeeter_deleter.py -s 1 -y -vv
```

L'output mostrerà in sequenza:

```
Archiving posts...
Downloading and archiving media...
Saving <hash>.jpeg
[...]
Found N posts to unlike.
Found N posts to unrepost.
Found N posts to delete.
Unliking N posts
Unliking: at://... by ...
[...]
Working... X% 0:MM:SS
```

La barra di progresso (`Working... X%`) mostra quanti elementi sono stati **tentati**, non quanti sono stati cancellati **con successo**. Questa distinzione è cruciale — vedi il capitolo successivo.

7. Problema noto: rate limit non gestito correttamente

Questo è il problema più insidioso incontrato durante il test reale, e vale la pena spiegarlo in dettaglio perché **la barra di progresso può arrivare al 100% anche se la cancellazione è quasi completamente fallita**.

7.1 Cosa succede

Bluesky impone un limite di **5.000 richieste API all'ora** per account. Nell'esecuzione test (2.600 elementi totali, di cui 1.486 immagini allegate ai post), la sola fase di **download media** ha consumato la maggior parte del budget orario prima ancora di iniziare la cancellazione vera.

Quando il rate limit si esaurisce, l'API risponde con **HTTP 429 (Rate Limit Exceeded)**. Lo script:

- logga l'errore tramite il modulo `logging` di Python, configurato per scrivere **su file** (`skeeter_deleter.log`), **mai a schermo**
- continua comunque a stampare `Deleting: [testo del post]` per ogni elemento, perché quella stampa avviene **prima** del tentativo di cancellazione, non dopo la sua conferma

Risultato: la barra di progresso arriva al 100%, ma il conteggio reale dei post rimasti è pressoché invariato.

7.2 Come diagnosticarlo

Non fidarti mai della sola barra di progresso. Verifica sempre con una query diretta all'API (paginando l'intero feed, perché il contatore `posts_count` del profilo è cache-ato e non affidabile in tempo reale):


```

cat > /tmp/verify.py << 'EOF'
from atproto import Client
import os
client = Client()
client.login(os.environ['BLUESKY_USERNAME'], os.environ['BLUESKY_PASSWORD'])

posts = 0
cursor = None
while True:
    feed = client.get_author_feed(os.environ['BLUESKY_USERNAME'], limit=100,
    cursor=cursor)
    posts += len([i for i in feed.feed if i.post.author.did == client.me.did])
    if not feed.cursor or len(feed.feed) == 0:
        break
    cursor = feed.cursor

likes = 0
cursor = None
while True:
    l = client.app.bsky.feed.get_actor_likes({'actor':
os.environ['BLUESKY_USERNAME'], 'limit': 100, 'cursor': cursor})
    likes += len(l.feed)
    if not l.cursor or len(l.feed) == 0:
        break
    cursor = l.cursor

print(f'Post rimanenti: {posts}')
print(f'Like rimanenti: {likes}')
EOF
python3 /tmp/verify.py

```

Se il numero non è sceso in modo significativo, controlla il file di log per confermare la causa:

```
grep -i "error" skeeter_deleter.log | head -20
```

Un errore tipo questo conferma il rate limit:

```

ERROR:An error occurred while unliking: Response(success=False, status_code=429,
content=XrpcError(error='RateLimitExceeded', message='Rate Limit Exceeded'),
headers={... 'ratelimit-remaining': '0', 'retry-after': '2128' ...})

```

Il campo `retry-after` (in secondi) o `ratelimit-reset` (timestamp Unix) ti dicono quando il limite si libera. Per convertire il timestamp in data leggibile:

```
date -r <timestamp>
```

8. Soluzione: script modificato senza download media

Se hai già un archivio scaricato da un tentativo precedente (o non ti interessa conservarlo), puoi risparmiare gran parte del budget di rate limit saltando la fase di download media, che è quella più dispendiosa in numero di chiamate.

8.1 Crea una copia dello script

```
cp skeeter_deleter.py skeeter_deleter_v2.py
```

8.2 Individua il blocco di download media

```
grep -n "def archive_repo" -A 30 skeeter_deleter.py
```

Cerca il ciclo che assomiglia a:

```
for cid in rich.progress.track(blob_cids):
    blob = self.client.com.atproto.sync.get_blob(params={'cid': cid, 'did': self.client.me.did})
    type = magic.from_buffer(blob, 2048)
    ext = ".jpeg" if type == "image/jpeg" else ""
    with open(f"archive/{clean_user_did}/_blob/{cid}{ext}", "wb") as f:
        if self.verbosity == 2:
            print(f"Saving {cid}{ext}")
        f.write(blob)
```

Annota i numeri di riga di inizio e fine di questo blocco (nel test erano le righe 367-374, ma possono variare a seconda della versione del repository).

8.3 Commenta il blocco

```
sed -i.bak '367,374s/^/#/' skeeter_deleter_v2.py
```

Sostituisci `367,374` con i numeri di riga reali trovati al passaggio precedente. Verifica il risultato:

```
sed -n '358,378p' skeeter_deleter_v2.py
```

Le righe del blocco di download dovrebbero ora iniziare con `#`.

8.4 Rilancia con la versione modificata

Prima verifica che il rate limit si sia resettato (attendi fino all'orario indicato da `ratelimit-reset` nel log), poi:

```
caffeinate -i python3 skeeter_deleter_v2.py -s 1 -y -vv
```

Questa volta la fase di archiviazione sarà quasi istantanea, lasciando molto più budget disponibile per le cancellazioni vere.

9. Verifica finale (non fidarsi della barra di progresso)

Anche dopo un'esecuzione “pulita”, **verifica sempre** con lo script del Capitolo 7.2. È normale che restino pochi elementi isolati (errori sporadici di rete, timeout, errori 500 del server) anche quando il rate limit non è più il problema principale.

Nel test reale, dopo la correzione del Capitolo 8, sono rimasti **5 post e 9 like** su oltre 2.600 elementi — un margine fisiologico, non un fallimento del processo.

10. Pulizia dei residui isolati

Se restano pochi elementi (dell'ordine delle decine), è più rapido cancellarli manualmente via script mirato piuttosto che rilanciare l'intero processo.

10.1 Identifica i post rimasti

```
cat > /tmp/list_remaining.py << 'EOF'
from atproto import Client
import os
client = Client()
client.login(os.environ['BLUESKY_USERNAME'], os.environ['BLUESKY_PASSWORD'])
feed = client.get_author_feed(os.environ['BLUESKY_USERNAME'], limit=100)
for item in feed.feed:
    if item.post.author.did == client.me.did:
        print(f'{item.post.record.created_at} - {item.post.uri}')
EOF
python3 /tmp/list_remaining.py
```

10.2 Cancellali specificamente

```
cat > /tmp/delete_remaining.py << 'EOF'
from atproto import Client
import os
client = Client()
client.login(os.environ['BLUESKY_USERNAME'], os.environ['BLUESKY_PASSWORD'])
uris = [
    # incolla qui gli URI trovati al passaggio 10.1
]
for uri in uris:
    try:
        client.delete_post(uri)
        print(f'OK: {uri}')
    except Exception as e:
        print(f'ERRORE su {uri}: {e}')
EOF
python3 /tmp/delete_remaining.py
```

10.3 Stessa logica per i like residui

```
cat > /tmp/unlike_remaining.py << 'EOF'
from atproto import Client
import os
client = Client()
client.login(os.environ['BLUESKY_USERNAME'], os.environ['BLUESKY_PASSWORD'])
cursor = None
count = 0
while True:
    likes = client.app.bsky.feed.get_actor_likes({'actor':
os.environ['BLUESKY_USERNAME'], 'limit': 100, 'cursor': cursor})
    for item in likes.feed:
        try:
            client.unlike(item.post.viewer.like)
            print(f'OK unlike: {item.post.uri}')
            count += 1
        except Exception as e:
            print(f'ERRORE su {item.post.uri}: {e}')
    if not likes.cursor or len(likes.feed) == 0:
        break
    cursor = likes.cursor
print(f'Totale unlike eseguiti: {count}')
EOF
python3 /tmp/unlike_remaining.py
```

Ripeti la verifica del Capitolo 9 finché entrambi i contatori non risultano a zero.

11. Ripristino del sistema

A operazione conclusa, ripristina le impostazioni di stand-by originali (annotate al passaggio 5.1):

```
sudo pmset -c sleep <valore-originale> disksleep <valore-originale> standby <valore-originale>
```

Verifica con `pmset -g` che i valori siano tornati quelli di partenza.

12. Considerazioni per account con molti più elementi

Il test descritto in questa guida riguardava circa 2.600 elementi totali. Per account con volumi molto maggiori (decine di migliaia), il problema del rate limit descritto al Capitolo 7 diventa **una certezza, non un’eventualità**: con un limite di 5.000 richieste/ora, qualunque combinazione di (post + repost + like + media) che superi quella soglia richiederà necessariamente più sessioni, distribuite nel tempo.

Alcuni approcci utili in questi casi:

- **Salta sempre il download media** se non ti interessa un backup locale (Capitolo 8), fin dal primo tentativo.
 - **Dividi il lavoro per intervalli di tempo**: modificando i filtri di raccolta dati (`gather_posts_to_delete`, `gather_likes`, `gather_reposts`) è possibile in linea di principio limitare l’elaborazione a un intervallo di date alla volta (es. un anno per sessione), restando sotto la soglia oraria. Questa modifica non è inclusa nello script originale e richiede un intervento manuale sul codice.
 - **Pianifica sessioni multiple**: se il volume supera abbondantemente le 5.000 unità, calcola in anticipo quante “ore di budget” ti serviranno e pianifica il lavoro su più giorni, verificando il progresso reale (Capitolo 7.2) tra una sessione e l’altra.
-

Limiti noti dello strumento

Per trasparenza verso chi userà questa guida:

- Il repository include un commento esplicito dello sviluppatore: *“the parameters are a mess, sorry, this is a to-fix”*.

- Il README stesso avverte: “*THIS CODE IS VERY DESTRUCTIVE. The maintainer assumes no liability or warranty for its use.*”
- Non esiste un meccanismo di retry/backoff automatico sugli errori 429: quando l’API restituisce un rate limit, lo script non aspetta e non riprova, semplicemente fallisce silenziosamente (loggando solo su file) e passa all’elemento successivo.
- Le fasi (archiviazione, unlike, delete) non sono eseguibili separatamente da riga di comando: sono tutte incluse nello stesso lancio, il che rende impossibile, senza modifiche al codice, riprendere da un punto intermedio in caso di interruzione.
- Il rilanciare lo script da capo rifà sempre l’intera archiviazione, anche se già presente localmente da un tentativo precedente.

Questi limiti sono gestibili con gli accorgimenti descritti in questa guida, ma è bene conoscerli prima di iniziare, specialmente su account con grandi volumi di dati.